

DecompGTI: Decomposed Graph Tool Instruction for Small Language Models

- Arpan Gupta (24110051)
- Buha Deep (24110082)
- Rathod Yuvraj (24110293)
- Solanki Viraj (24110348)



Problem Statement

Small Language models struggle with graph reasoning when asked to solve tasks directly

Common failure points:

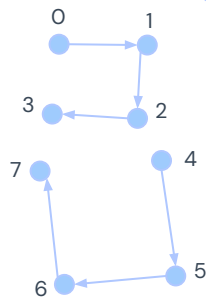
- Incomplete graph reconstruction
- Wrong tool or algorithm selection
- Missing or malformed parameters
- Incorrect final answers even when the reasoning is mostly right

Example of such Problem

Question:

Node 0 is connected to node 1 (weight: 5).
Node 1 is connected to node 2 (weight: 5).
Node 2 is connected to node 3 (weight: 5).
Node 4 is connected to node 5 (weight: 5).
Node 5 is connected to node 6 (weight: 5).
Node 6 is connected to node 7 (weight: 5).

Question: Calculate the distance of the shortest path from node 0 to node 7.



Answer:

Llama-3-8B-Instruct

the shortest distance from
node 0 to node 7 is 15



Ours

No path from 0 to 7



Core Idea

1. Reframe graph reasoning as structured tool orchestration
2. The model predicts a JSON plan instead of direct answer
3. A deterministic runtime executes the selected graph algorithm

System Overview



Graph
extraction



Tool
Identification



parameter
extraction



Json Validation
and routing



Algorithm
execution

Tool Layer

Tool	Purpose
bfs	Visits nodes level by level from a start node
dfs	Traverses as deep as possible before backtracking
dijkstra_shortest_path	Finds the minimum-cost path between source and target nodes
maximum_flow	Computes the largest possible flow from source to sink
bipartite_maximum_matching	Finds the largest set of non overlapping matches across two partitions
topological_sort	Produces a valid dependency order for nodes in a DAG
cycle_detection	Checks whether the graph contains at least one cycle
minimum_spanning_tree	Builds a minimum total weight tree containing all reachable nodes

Data Generation

GraphInstruct
generation[2]



Format
conversion



DecompGTI
training sample

- We generated a total of **15000** samples spanning **different graph reasoning tasks** across three size bands, and **fine tuned** the model on this full dataset

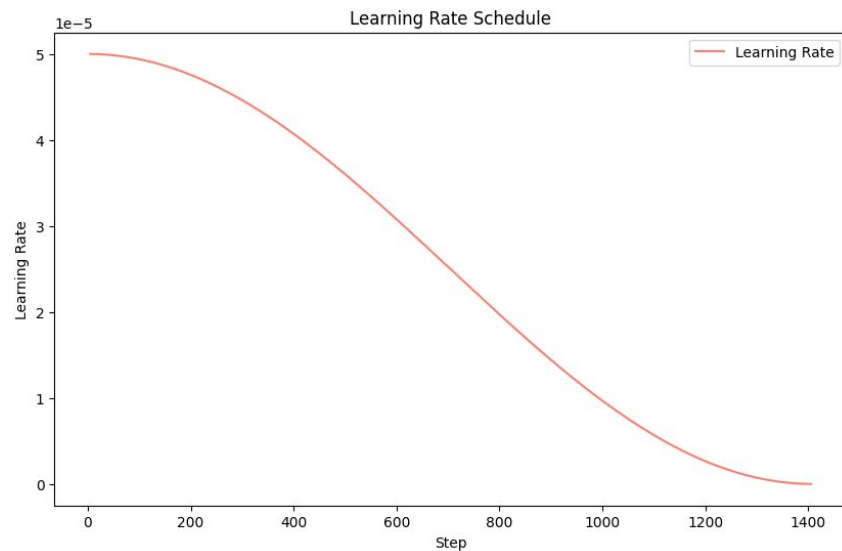
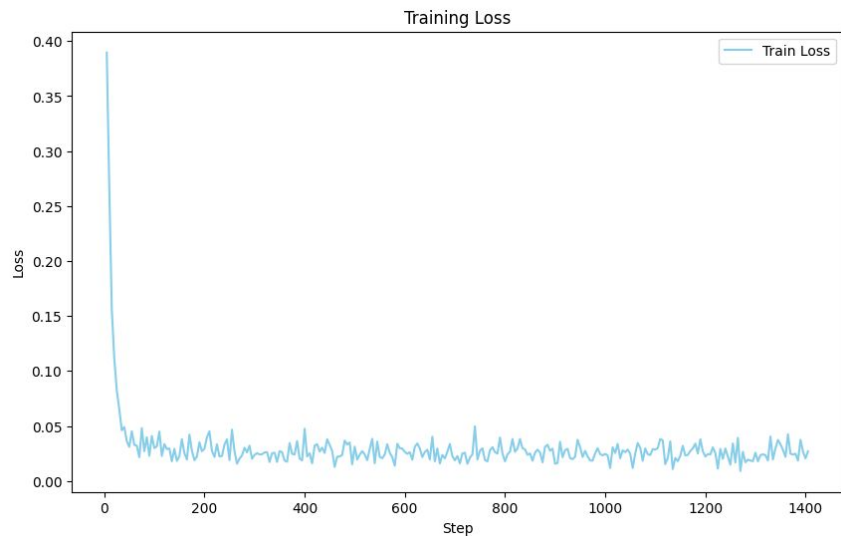
Sample target output:

```
{
  "step1_graph_extraction": {
    "adjacency_list": "{0: [(1, weight:8), (2, weight:7)], 1: [(0, weight:5)]}",
    "directed": false
  },
  "step2_tool_name": "shortest_path",
  "step3_tool_parameters": {
    "source": 0,
    "target": 2
  }
}
```

Model Fine-Tuning Setup

- Base model: **Qwen2.5-7B**[\[5\]](#)
- Method: **LoRA**
- Objective: **3-step structured JSON output**
- Framework: **LLaMaFactory**[\[3\]](#)
- Epochs: **3**
- Batch size: **16**
- Learning rate: **5e-5**

Model Training Progress



Benchmark Generation

- Generated benchmark graphs with NetworkX^[4] using the same task families as the training step
- Built evaluation samples for the size:
 - **Mini:** 5-7 Nodes
 - **Small:** 8-15 Nodes
 - **Medium:** 16-25 Nodes
 - **Large:** 26-50 Nodes
- The generated samples store the graph description, adjacency list, question, task type, graph size, expected tool, expected parameters, and expected answer

Models to Compare

Compare DecompGTI against baseline and frontier models under the same benchmark settings

Qwen 2.5-7B-Instruct[\[6\]](#)

DecompGTI
Fined-tuned model[\[5\]](#)

Mistral-7B-Instruct[\[7\]](#)

Llama-3-8B-Instruct[\[8\]](#)

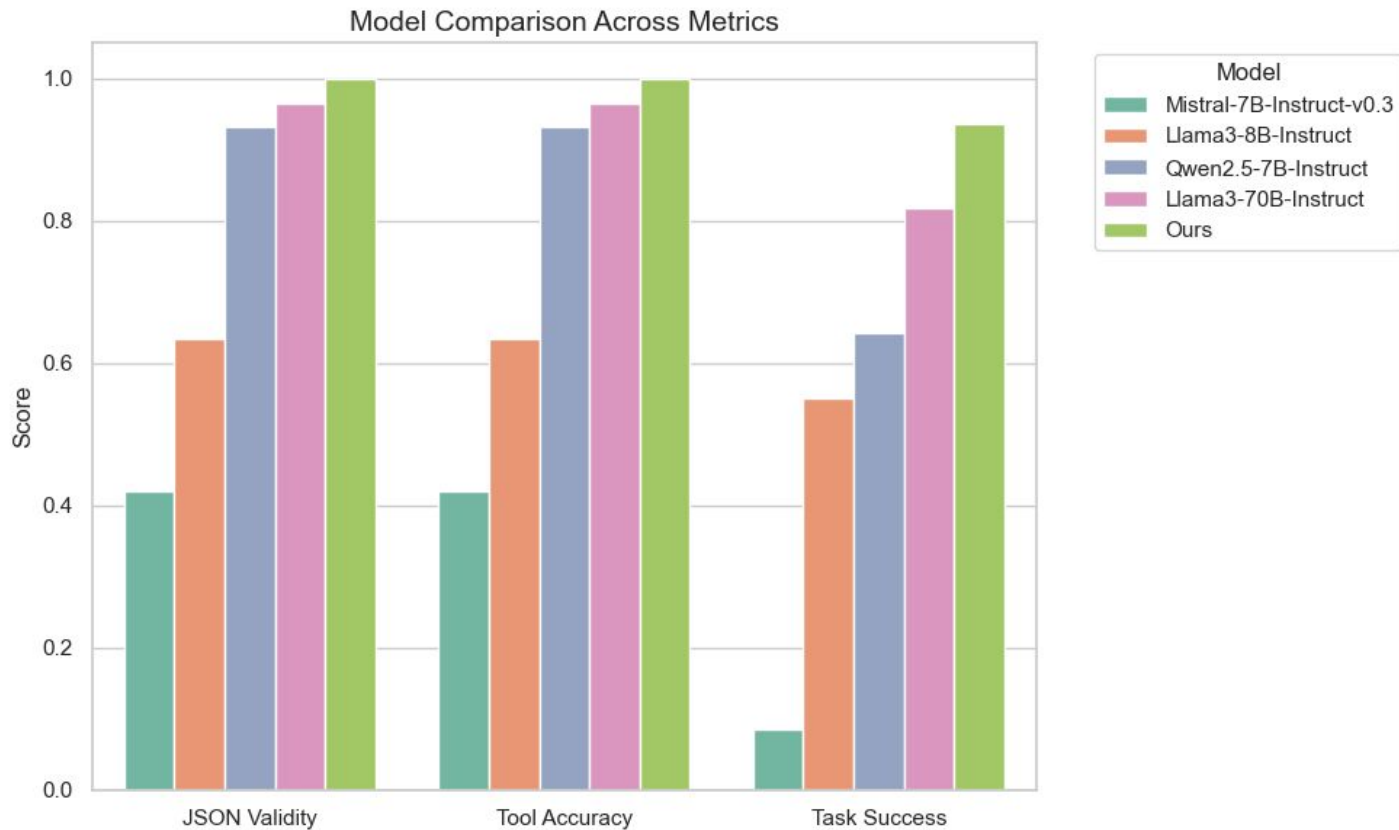
Llama-3-70B-Instruct[\[9\]](#)

All Experiments on this models are performed on the same task set, same size tiers(mini/small/medium/large), same evaluation metrics

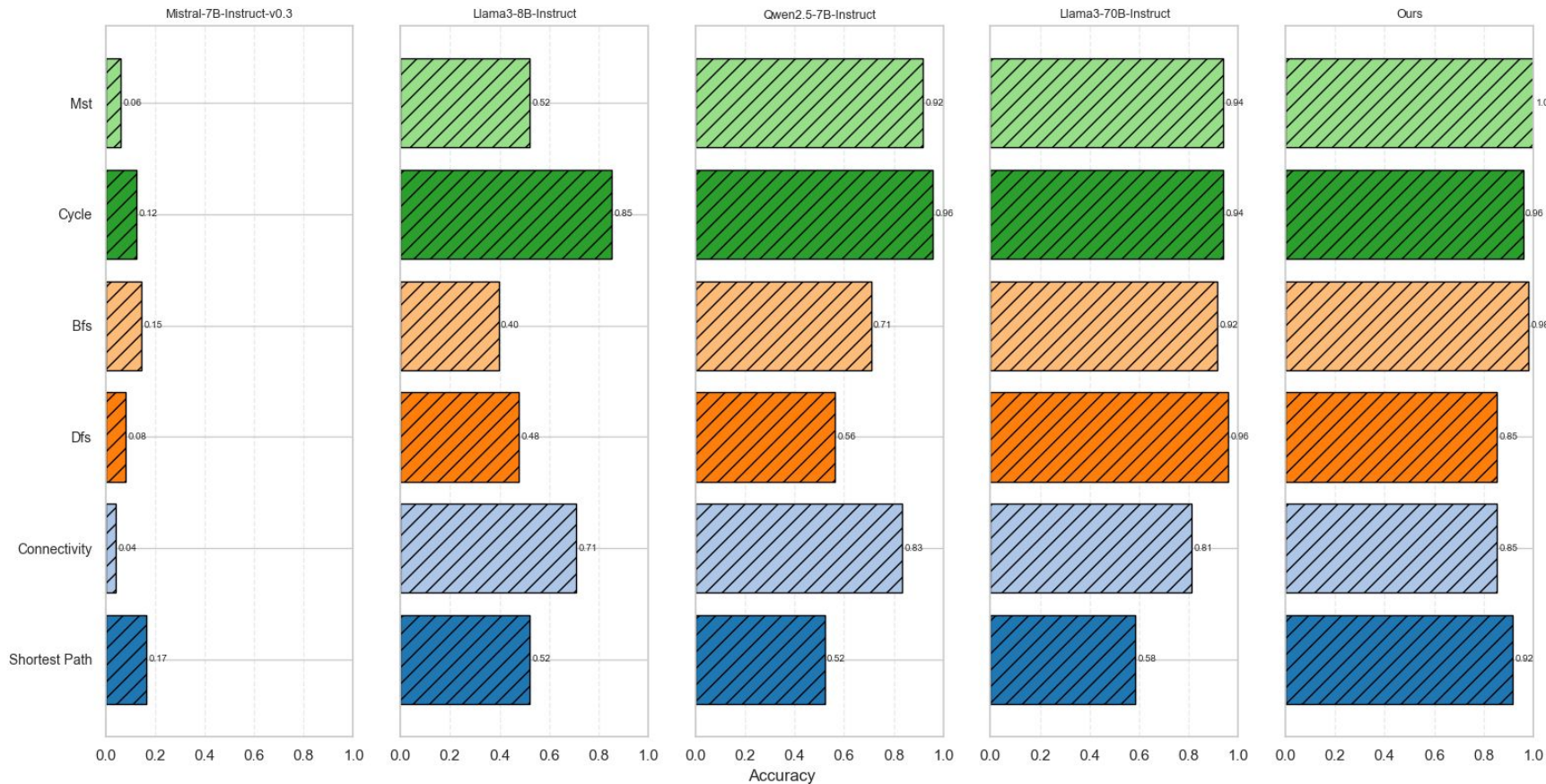
Evaluation Metrics

1. **JSON Validity Rate:** how often the model output is a parseable and scheme compliant JSON plan
2. **Tool Identification Accuracy:** how often the predicted tool matches the expected algorithm
3. **Parameter Extraction Precision, Recall, and F1:** how correctly and completely the model recovers required tool arguments
4. **Adjacency Extraction F1:** how well the reconstruction graph structure matches the reference graph
5. **Task Success Rate:** how often end to end execution returns the correct final answer

Experimental Results

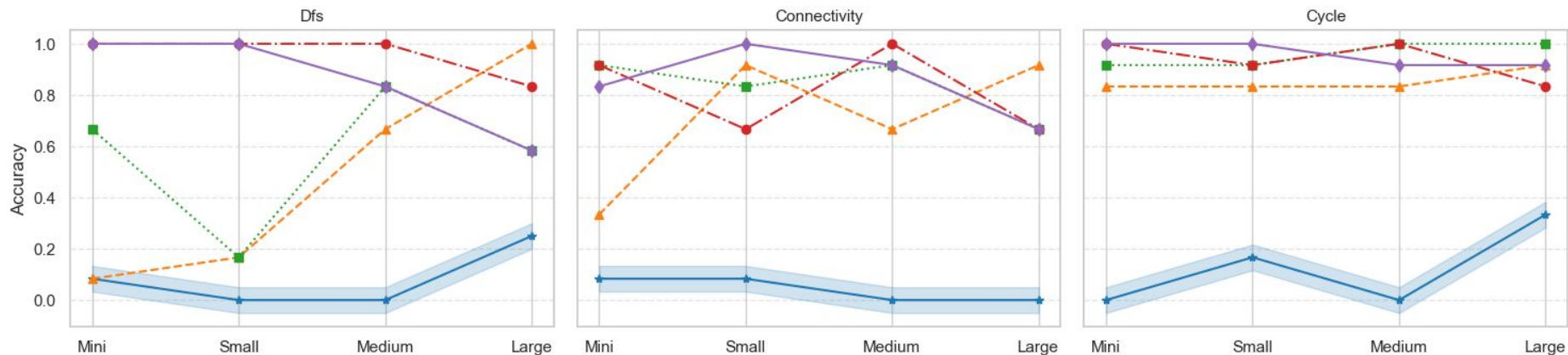


Experimental Results



Experimental Results

Mistral-7B-Instruct-v0.3 Llama3-8B-Instruct Qwen2.5-7B-Instruct Llama3-70B-Instruct Ours



Limitations and Future Work

1. Compare our model with closed-source model
2. Improve parameter recovery for harder tasks
3. Benchmark model on very large graphs (100-200 Nodes)
4. Add more tasks on complex graph questions
5. Train our models on unweighted graph problems too.

Conclusion

- **DecompGTI** enables graph reasoning by converting problems into **structured decomposition + deterministic tool execution**
- Provides a **complete pipeline**: data generation, fine-tuning, tool routing, execution, and evaluation.
- Archives **high accuracy in structured generation and tool selection**
- **Next step**: improve end-to-end task success via better evaluation coverage and expanded benchmarks

References

1. <https://github.com/yuvraj-rathod-1202/DecompGTI>
2. [GraphInstruct: graph reasoning task generation and evaluation resources.](#)
3. [LLaMA-Factory: efficient fine-tuning toolkit for large language models.](#)
4. <https://networkx.org/>
5. <https://huggingface.co/Qwen/Qwen2.5-7B>
6. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>
7. <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>
8. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>
9. <https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct>

The image features a light gray background with several overlapping hexagonal shapes in various shades of blue and cyan. These shapes are positioned in the corners: top-left, top-right, and bottom-right. The central text "Thank You" is rendered in a bold, dark blue, sans-serif font.

Thank You